Consider a function of two variables

$$f(x,y) = 2x^3y^2$$

- Let differentiate "f" w.r.t "x" keeping "y" constant we want to find the rate of change of "f" w.r.t "x"

$$f_x'(x,y) = 6x^2y^2$$

- The rate of change of "f" w.r.t "y" keeping x constant can be :

$$f_y'(x,y) = 4x^2y$$

$$f_x'(x,y) = \lim_{h\to 0} \frac{f(x+h,y) - f(x,y)}{h}$$

$$f_y'(x,y) = \lim_{h\to 0} \frac{f(x,y+h) - f(x,y)}{h}$$

This can also be written as

$$f_x'(x,y) = f_x(x,y) = \frac{\partial f}{\partial x} = \frac{\partial}{\partial x}(f(x,y)) = z_x$$

$$= \frac{\partial z}{\partial x} = D_x f$$

and

$$f_y'(x,y) = f_y(x,y) = \frac{\partial f}{\partial y} = \frac{\partial}{\partial y}(f(x,y)) = z_y = \frac{\partial z}{\partial y} = D_y f$$

CamScanner

If function "f" is only of single variable "x"

$$f(x) \Longrightarrow f'(x) = \frac{df}{dx}$$ → Simple derivative

$$f(x,y) \Longrightarrow f'_x(x,y) = f_x(x,y) = \frac{\partial f}{\partial x}$$ → partial derivative or 1st order partial derivative

$$\hookrightarrow f'_y(x,y) = f_y(x,y) = \frac{\partial f}{\partial y}$$

example

$$f(x,y) = x^4 + 6\sqrt{y} - 20$$

$$f_x(x,y) = 4x^3$$

$$f_y(x,y) = \frac{3}{\sqrt{y}}$$

Chain Rule

single variable: $y = f(x)$ and $x = g(t)$

$$\frac{dy}{dt} = \frac{dy}{dx} \cdot \frac{dx}{dt}$$

What if we have multiple variables.

$$z = f(x,y),\ x = g(t),\ y = h(t)$$

$$\frac{dz}{dt} = \frac{\partial f}{\partial x} \cdot \frac{dx}{dt} + \frac{\partial f}{\partial y} \cdot dy/dt$$

total derivative of function z w.r.t 't' is the sum of the product of the partial derivate times the derivative of the variable w.r.t 't'.

Case 1

$f(v_1, v_2)$, $v_1 = g(t)$, $v_2 = h(t)$

total derivative of "f" w.r.t "t" is sum of the product of partial derivative of "f" w.r.t "v_1" and the derivative of "v_1" w.r.t "t" & vice versa for other variable "v_2".

$$\frac{df}{dt} = \frac{\partial f}{\partial v_1} \cdot \frac{dv_1}{dt} + \frac{\partial f}{\partial v_2} \cdot \frac{dv_2}{dt}$$

v_1, v_2 → Intermediate variables, that both dependent on t. $v_1 = g(t)$, $v_2 = h(t)$

z → the dependent variable ($z = f(x, y)$)

t → the only independent variable.

Case 2

Lets say that

$$z = f(x, y), \quad x = g(t), \quad y = h(J)$$

Here "z" is a function of two independent variables "t" and "J" we are no longer interested in total derivative of z w.r.t "t" i.e $\frac{dz}{dt}$ but rather on partial derivatives of z w.r.t "t" and "J".

$$\frac{\partial z}{\partial t} = \frac{\partial f}{\partial x} \cdot \frac{dx}{dt}$$

$$\frac{\partial z}{\partial J} = \frac{\partial f}{\partial y} \cdot \frac{dy}{dJ}$$

Now total differential dz is

$$dz = \frac{\partial z}{\partial t} dt + \frac{\partial z}{\partial J} dJ$$

$$dz = \left(\frac{\partial f}{\partial x} \cdot \frac{dx}{dt}\right) dt + \left(\frac{\partial f}{\partial y} \cdot \frac{dy}{dJ}\right) dJ$$

Case 3

lets say that

$$x = f(u, y) \quad , y = g(x)$$

total derivative dx becomes $\frac{dx}{du}$

$$\frac{dx}{du} = \frac{\partial f}{\partial u} \cdot \frac{du}{du} + \frac{\partial f}{\partial y} \cdot \frac{dy}{du}$$

$$\frac{dz}{du} = \frac{\partial f}{\partial u} + \frac{\partial f}{\partial y} \frac{dy}{du}$$

Case 4

$$z = f(u, y), \quad u = g(s, t), \quad y = h(s, t)$$

to find total derivative, we would Compute

$\frac{\partial z}{\partial s}$ & $\frac{\partial z}{\partial t}$

$$\frac{\partial z}{\partial s} = \frac{\partial f}{\partial u} \cdot \frac{\partial u}{\partial s} + \frac{\partial f}{\partial y} \cdot \frac{\partial y}{\partial s}$$

$$\frac{\partial z}{\partial t} = \frac{\partial f}{\partial u} \cdot \frac{\partial u}{\partial t} + \frac{\partial f}{\partial y} \cdot \frac{\partial y}{\partial t}$$

$$dz = \frac{\partial z}{\partial s} ds + \frac{\partial z}{\partial t} dt$$

CamScanner

General case / multivariable chain rule

z is a function of "n" variables $x_1, x_2, \ldots, x_n$ & each of these variables $(x_i, i=1,\ldots n)$ are in turn functions of m variables $t_1, t_2, \ldots, t_m$. Then to find partial derivative of "z" w.r.t any of independent variable t_i, $i=1,2,\ldots m$

$$\frac{\partial z}{\partial t_i} = \frac{\partial z}{\partial x_1}\frac{\partial x_1}{\partial t_i} + \frac{\partial z}{\partial x_2}\frac{\partial x_2}{\partial t_i} + \cdots + \frac{\partial z}{\partial x_n}\frac{\partial x_n}{\partial t_i} \quad \text{(A)}$$

total derivative

$$dz = \left(\sum_{i=1}^{m} \frac{\partial z}{\partial t_i} dt_i\right) \rightarrow \text{(B)}$$

$$dz = \left(\sum_{j=1}^{n} \frac{\partial z}{\partial x_j}\frac{\partial x_j}{\partial t_1}\right) dt_1 + \left(\sum_{j=1}^{n} \frac{\partial z}{\partial x_j}\frac{\partial x_j}{\partial t_2}\right) dt_2 + \cdots + \left(\sum_{j=1}^{n} \frac{\partial z}{\partial x_j}\frac{\partial x_j}{\partial t_m}\right) dt_m$$

↳ (C)

Neural Network :- take "z" as final output, $x_1, x_2 \ldots x_n$ as neurons in a hidden layer. & $t_1, t_2, \ldots t_m$ are weights or bias as independent variables

- multi variable chain rule
- backpropagation
- Gradient descent

Training a neural network is actually solving (A) a million times.

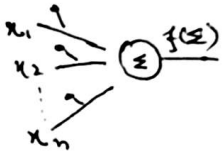
Neural Network

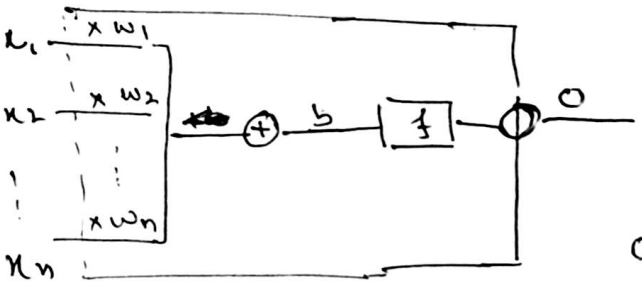
1. Perceptron :- the basic building block. a single neuron.

For an input vector "x", the neuron calculates

a. Linear combination: $a = \sum(w_i x_i) + b$

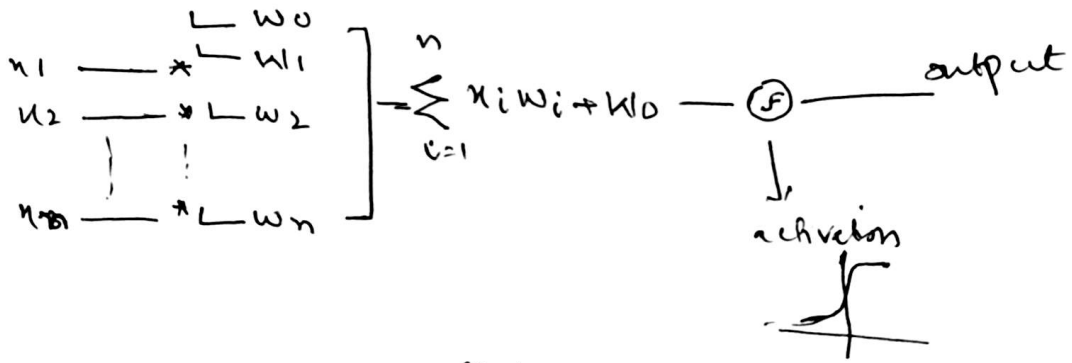
b. Activation: $h = \sigma(a)$

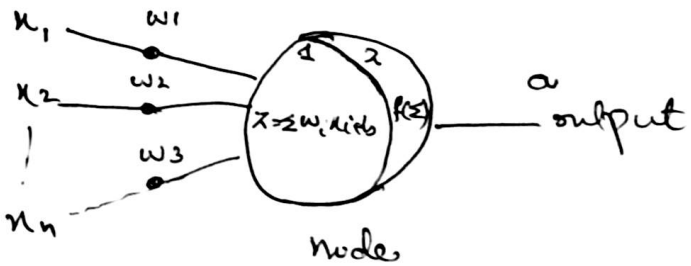




$$o = f\left(\sum_{i=1}^{n} x_i \cdot w_i + b\right)$$

when the





does two jobs (linear aggregation at step1)

and non-linear activation at step2.

Without step2, a nn with 100 layers would be a giant linear equation. Nolearning

The non-linear activation allows the nn to learn complex patterns (zig-zag, curve etc).

The bias allows the activation function to shift left or right (flexibility).

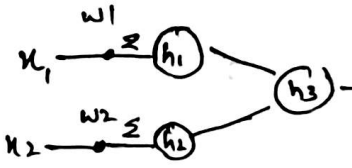
2. The Architecture, Layers?

NN is organized into input layer, Hidden layers & output layer.

- Input layer:- Holds raw data (image pixel values)
- Hidden layers:- feature extractors
  - The first hidden layer might detect simple edges
  - The second h. layer might detect shapes (circle, square etc)
  - The third might detect complex objects like eyes, wheels etc.

More layers (for deep learning hidden layer shall be atleast 2) allow nn to learn complex hierarchy of features.

3. x_1 & x_2 are two features & we want to predict whether it will sell or not ($y=1$, or 0).



$$J = \frac{1}{2}(\hat{y} - y)^2$$

4. Back propagation

We want to know how much the loss (J) changes if we tweak a weight in the first layer.

By multivariable chain, the error flows backward.

$$\frac{\partial J}{\partial w_{initial}} = \frac{\partial J}{\partial y'} \cdot \frac{\partial y'}{\partial h_{hidden}} \cdot \frac{\partial h_{hidden}}{\partial a_{hidden}} \cdot \frac{\partial a_{hidden}}{\partial w_{initial}}$$

(4A)

Compound change

$$J(y'(h(a(w))))$$

- a change in w causes change in a, which is triggered all the way up to J.

- And if for a reduced "J" w.r.t tweaking w tiny change in initial will trigger a change in all hidden layers from "J" backward until input layer where w_i is changed.

$$\frac{\partial J}{\partial w_i} = \sum_{paths} \prod_{product} \text{derivatives along path (4A)}$$

if we want to find a gradient for a weight in the first layer, we multiply along the chain (the path) and then sum all the different chains that lead to the output.

Product(A·B·C) → depth going through layers

Sum(A+B) → breadth, multiple neurons in same layer

Weight update

∇J is called gradient.

$$w_{new} = w_{old} - \alpha(\nabla J)$$

$$w_{new} = w_{old} - \alpha \cdot \frac{\partial J}{\partial w}$$

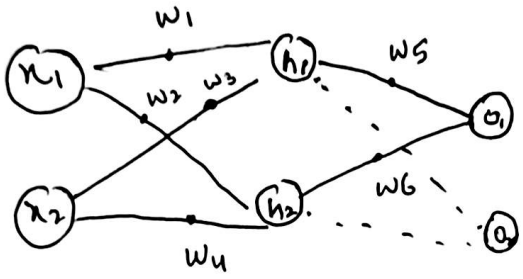
$x_1 = 0.05, x_2 = 0.10$

$w_1, w_2, w_3, w_4 \begin{bmatrix} 0.15 \\ 0.20 \\ 0.25 \\ 0.30 \end{bmatrix}$

$w_5, w_6 \begin{bmatrix} 0.40 \\ 0.45 \end{bmatrix}$

w_7 w_8 for o_2 is ignored for simplicity

$w_7, w_8 \begin{bmatrix} 0.40 \\ 0.45 \end{bmatrix}$



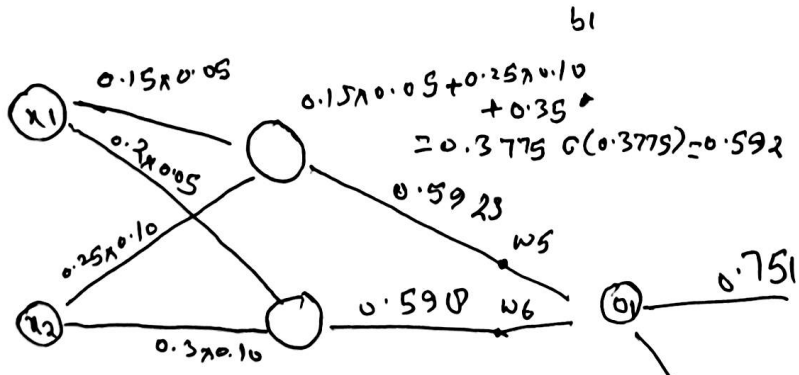
$b_1 = 0.35$

$b_2 = 0.60$

Target $y = 0.01$

$\sigma(x) = \frac{1}{1+e^{-x}}$

$\sigma'(x) = \sigma(x)(1-\sigma(x))$



$0.25 \times 0.05 + 0.3 \times 0.10 + 0.35$

$= 0.3925$

$= \sigma(0.3925)$

$= 0.5909$

$net_{o1} = w_5 \cdot out_{h_1} + w_6 \cdot out_{h_2} + b_2$

$= 1.1058$

$= \sigma(1.1058) = 0.75$

error $E_{total} = \frac{1}{2}(y - \hat{y}) = \frac{1}{2}(0.01 - 0.75)^2$

$= 0.2748$

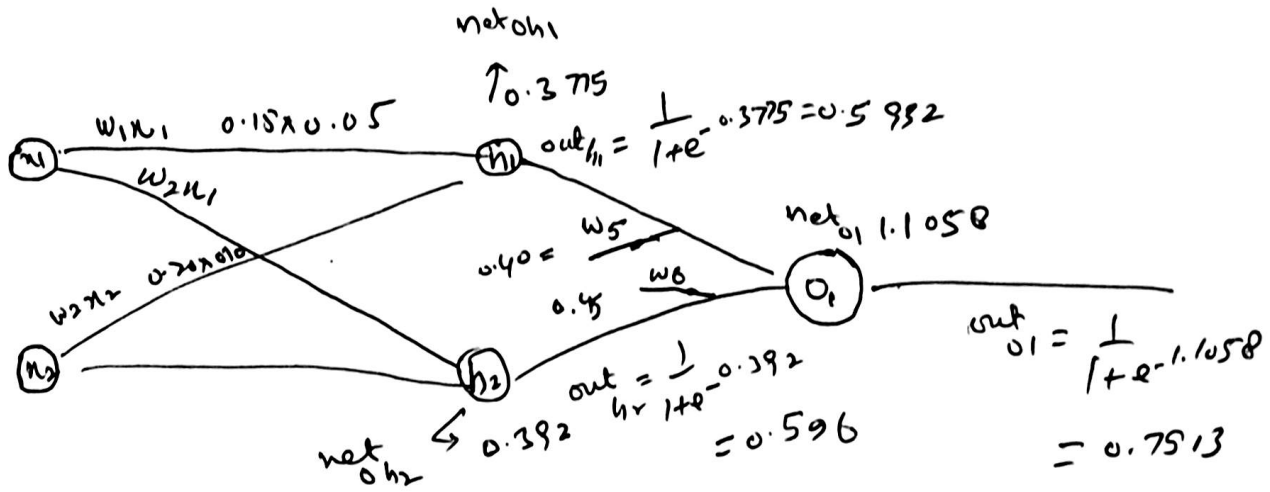
Backward Propagation

need to find

$$\frac{\partial E_{total}}{\partial w}$$

this is where chain rule happens.

lets re write what were the building blocks in forward pass.



$$E_{Total} = \frac{1}{2}(\hat{Y} - Y)^2 = \frac{1}{2}(0.7513 - 0.01)^2 = 0.2748$$

We want to find $\frac{\partial E_{total}}{\partial w}$.

* W5 update.

(1) Error changes w.r.t output

$$\frac{\partial E_{Total}}{\partial out_{o1}} = \hat{y} - y = 0.7513 - 0.01 = 0.743$$

(2) output changes w.r.t to Net:

$$\frac{\partial out_{o1}}{\partial net_{o1}} = out_{o1}(1 - out_{o1}) = 0.7513(1 - 0.7513) = 0.1868$$

(3) Net change w.r.t Weight $\frac{\partial net_{o1}}{\partial w_5} = out_{h1} = 0.5932$

The gradient for w_5 is

$$\frac{\partial E_{total}}{\partial w_5} = \frac{\partial E_{total}}{\partial out_{o1}} \cdot \frac{\partial out_{o1}}{\partial net_{o1}} \cdot \frac{\partial net_{o1}}{\partial w_5}$$

$$= 0.7413 * 0.1868 * 0.5932 = 0.0821$$

$$w_{5new} = w_{5old} - \alpha \text{ gradient}$$

$$= 0.40 - 0.5 * 0.0821 = 0.3589$$

updating hidden weight w_1

$$\frac{\partial E_{total}}{\partial w_1} = \frac{\partial E_{total}}{\partial out_{h1}} \cdot \frac{\partial out_{h1}}{\partial net_{h1}} \cdot \frac{\partial net_{h1}}{\partial w_1}$$

$\frac{\partial E_{total}}{\partial out_{h1}}$ is the error from output neuron scaled by w_5

$$\frac{\partial E_{total}}{\partial out_{h1}} = 0.7413 * 0.1868 * w_5$$

$$= 0.1384 * 0.40 = 0.0553$$

1. $\frac{\partial out_{h1}}{\partial net_{h1}}$ (sigmoid derivative) $= 0.5932(1-0.5932)$

$$= 0.2413$$

2. $\frac{\partial net_{h1}}{\partial w_1}$ (input) $x_1 = 0.05$

Gradient for w_1 is

$$\frac{\partial E_{total}}{\partial w_1} = 0.0553 * 0.2413 * 0.05 = 0.00066$$

$$w_{1new} = 0.15 - 0.5 * 0.00066 = 0.1496$$

[redacted]